Luma API vs Python SDK

Let's review the video generation component of project Reachy_Local_08.4.2. Recall Luma AI provides the video generation model used by the web app.
Details about LumaAI are available here:
https://lumalabs.ai/, https://lumalabs.ai/api, https://docs.lumalabs.ai/docs/api, https://docs.lumalabs.ai/docs/video-generation, https://docs.lumalabs.ai/docs/python

Explain how Luma's API and Python SDK are used. I don't understand the distinction between the two, especially when used for video generation. Is the Python SDK the Python version of the API?

Thought for 18s >

Short version:

- **The “API” is the actual door into Luma’s servers (HTTP endpoints).**
- **The Python SDK is just a nice Python wrapper around those same endpoints.**
  Same backend, two different ways of talking to it.

Let’s unpack it in Reachy terms.

---

1. What the Luma *API* actually is

The Dream Machine / Luma API is just a set of HTTPS endpoints like:

- `POST https://api.lumalabs.ai/dream-machine/v1/generations` (or `/generations/video`) – start a generation
- `GET  https://api.lumalabs.ai/dream-machine/v1/generations/{id}` – poll status
- Then you download the final `video.mp4` from the URL they give you.
  Dream Machine … +3

The basic pattern is:

1. **Create a generation**

   ```
   curl --request POST \
        --url https://api.lumalabs.ai/dream-machine/v1/generations \
        --header 'authorization: Bearer luma-xxxx' \
        --header 'content-type: application/json' \
        --data '{
          "prompt": "A happy person smiling at the camera",
          "model": "ray-2",
          "resolution": "720p",
          "duration": "5s"
        }'
   ```

   That returns JSON with an `id`, `state` (`queued`, `dreaming`, `completed`, etc.) and metadata.

2. **Poll for completion**

   ```
   curl \
     https://api.lumalabs.ai/dream-machine/v1/generations/<id> \
     -H 'authorization: Bearer luma-xxxx'
   ```

3. **Download the video**

   Once `state == "completed"`, the response contains a URL; you `GET` that URL and save it as `video.mp4`. Dream Machine… +1

That's the **bare-metal version**.
Everything else (Python SDK, JS SDK, n8n nodes) is just "helping you do these same HTTP calls."

For Reachy_Local_08.4.2, your requirements literally say:

> *Web UI triggers synthetic generation (via Luma API).*

That's API-level language: "we will call those HTTP endpoints somewhere in the stack."

---

2. What the Python SDK does on top

Luma's Python SDK (`lumaai-python`) is a **Python library that talks to that same REST API for you**. From their README:

> *“The LumaAI Python library provides convenient access to the LumaAI REST API from any Python 3.8+ application… offers sync and async clients powered by httpx.”* GitHub

Docs call it “Dream Machine Python SDK” and show exactly the same flow: create → get status → download. Dream Machine … +1

So instead of hand-rolling `requests.post` / `requests.get`, you do something like:

```
from lumaai import LumaAI
import time
import requests
import os

client = LumaAI(auth_token=os.environ["LUMA_API_KEY"])

# 1. Kick off text-to-video
generation = client.generations.create(
    prompt="A person smiling and waving at the camera",
    model="ray-2",
    resolution="720p",
    duration="5s",
)

gen_id = generation.id

# 2. Poll until it's done
while generation.state not in ("completed", "failed"):
    time.sleep(5)
    generation = client.generations.get(gen_id)

if generation.state == "failed":
    raise RuntimeError(f"Luma generation failed: {generation.failure_reason}")

# 3. Download the video
video_url = generation.assets["video"]
resp = requests.get(video_url, stream=True)
with open("happy_001.mp4", "wb") as f:
    f.write(resp.content)
```

Under the hood, this is still:

- `POST /generations`

- `GET /generations/{id}`
- `GET video_url`

The SDK just:

- Handles **base URL + auth headers** for you.
- Gives you **Python objects** instead of raw dicts.
- Has **typed params** and sometimes built-in helpers (sync/async, pagination, etc.). Dream Machine ... +2

So to your question:

> *Is the Python SDK the Python version of the API?*

Close, but important nuance:

- The **API** = the public contract exposed over HTTP (what exists on Luma's side).
- The **Python SDK** = a Python *client* for that API (what you install on your side).

You *never* "choose API *or* SDK" in the sense of different backends.
You're always using the API. The only choice is **"raw HTTP vs helper library."**

---

3. Side-by-side: raw HTTP vs Python SDK

Just to anchor it, here's a minimal comparison for *text → video*.

A) Direct REST (no SDK, e.g. in an n8n HTTP Request node or plain requests)

```
import os, time, requests

API_KEY = os.environ["LUMA_API_KEY"]
BASE_URL = "https://api.lumalabs.ai/dream-machine/v1"

# 1. Create generation
resp = requests.post(
    f"{BASE_URL}/generations",
    headers={
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json",
        "Accept": "application/json",
    },
    json={
```

```
        "prompt": "A cheerful person laughing",
        "model": "ray-2",
        "resolution": "720p",
        "duration": "5s",
    },
)
resp.raise_for_status()
data = resp.json()
gen_id = data["id"]

# 2. Poll status
while data["state"] not in ("completed", "failed"):
    time.sleep(5)
    resp = requests.get(
        f"{BASE_URL}/generations/{gen_id}",
        headers={"Authorization": f"Bearer {API_KEY}"},
    )
    resp.raise_for_status()
    data = resp.json()

if data["state"] == "failed":
    raise RuntimeError(data)

video_url = data["assets"]["video"]

# 3. Download
video_resp = requests.get(video_url, stream=True)
with open("happy_002.mp4", "wb") as f:
    f.write(video_resp.content)
```

B) Same thing via Python SDK

```
from lumaai import LumaAI
import os, time, requests

client = LumaAI(auth_token=os.environ["LUMA_API_KEY"])

generation = client.generations.create(
    prompt="A cheerful person laughing",
    model="ray-2",
```

```
    resolution="720p",
    duration="5s",
)

while generation.state not in ("completed", "failed"):
    time.sleep(5)
    generation = client.generations.get(generation.id)

if generation.state == "failed":
    raise RuntimeError(generation)

video_url = generation.assets["video"]
video_resp = requests.get(video_url, stream=True)
with open("happy_002.mp4", "wb") as f:
    f.write(video_resp.content)
```

Same semantics, less plumbing.

---

4. How this plugs into Reachy_Local_08.4.2

Your spec says:

- **FR-001**: "Web UI triggers synthetic generation (via Luma API). Users accept/reject clips, labeling 'happy' or 'sad'."
- AGENTS file: synthetic videos from external APIs must be labeled and logged.

There are two sane ways to wire this in your stack:

Option 1 – Pure API from n8n / Streamlit (no SDK)

- **Streamlit UI** → hits **FastAPI "Generate" endpoint** on Ubuntu 1.
- That endpoint uses `requests` to call Luma's REST API directly (pattern A above).
- Once the video is downloaded, the service:
  - Saves it to `/videos/temp/…`
  - Registers it in Postgres (Ingest Agent)
  - Emits the appropriate `ingest.completed` / `video.generated` event into n8n.

This matches the "via Luma API" language literally and keeps dependencies minimal.

Option 2 – Tiny "Luma client" service using the Python SDK

Same flow, but inside that FastAPI service you use the SDK instead of hand-rolled HTTP:

- Easier to maintain if you later:
  - Add **image-to-video** or **extend video** features. Dream Machine… +1
  - Use advanced options like `concepts`, `keyframes`, `loop`, `aspect_ratio`. Dream Machine… +1
- You still treat it externally as "Luma API" in the architecture doc; implementation detail is just "using the official Python client."

Either way, from Reachy's point of view you get:

1. A short MP4 in `/media/project_data/reachy_emotion/videos/temp/…`
2. Postgres row with `generator='luma'`, prompt, model, etc.
3. That video then goes through Labeling → Promotion → Training, as already defined.

---

5. How to think about it going forward

If you're:

- **Working inside Python (FastAPI microservice)** → SDK is *usually* nicer.
- **Working in low-code / other languages (n8n HTTP node, Bash, etc.)** → call the HTTP API directly.
- **Worried about lock-in or debugging** → starting with raw HTTP teaches you exactly what's happening on the wire; you can always swap to the SDK later.

So: the Python SDK isn't a *different* API, it's your "Python-shaped handle" on the same Dream Machine API your requirements already talk about.

If you want, next step we can sketch the exact FastAPI endpoint for "Generate via Luma, stash to `/videos/temp`, emit Ingest event" and decide whether to do it with `requests` or the SDK.